

Chapter 13: Data Storage Structures

1 Employee File (Figure 13.3)

Record	ID	Name	Department	Salary
0	10101	Srinivasan	Comp. Sci.	65000
1	12121	Wu	Finance	90000
2	15151	Mozart	Music	40000
3	98345	Kim	Elec. Eng.	80000
4	32343	El Said	History	60000
5	33456	Gold	Physics	87000
6	45565	Katz	Comp. Sci.	75000
7	58583	Califieri	History	62000
8	76543	Singh	Finance	80000
9	76766	Crick	Biology	72000
10	83821	Brandt	Comp. Sci.	92000

This table represents a fixed-length employee record file, where each entry is identified by its record number.

2 Q&A Discussion

2.1 Deletion Strategies

Question

Consider the deletion of record 5 (Gold, Physics, 87000). Compare the relative merits of the following techniques:

- (a) Move record 6 to the space occupied by record 5, and move record 7 to the space occupied by record 6, and so on.
- (b) Move record 7 to the space occupied by record 5.

(c) Mark record 5 as deleted, and move no records.

Answer

(a) Move record 6, then 7, etc.

- **Advantage:** Maintains compact storage and physical order.
- **Disadvantage:** Expensive if many records must be shifted; inefficient for large files.

(b) Move record 7 into record 5's space

- **Advantage:** Requires only one move.
- **Disadvantage:** Breaks logical ordering, making sequential scans less consistent.

(c) Mark record 5 as deleted (no movement)

- **Advantage:** Very efficient—no shifting required.
- **Disadvantage:** Leaves unused space unless a free-list reuse mechanism is implemented.
- **Usage:** Common in DBMS—deleted slots are later reused for inserts.

Conclusion: - Method (a): Suitable for small, ordered files. - Method (b): Minimizes moves but disrupts order. - Method (c): Most efficient; widely adopted with free-space management.

2.2 File Updates with Free List (Figure 13.4)

Question

Show the structure of the file of Figure 13.4 after each of the following steps:

- (a) Insert (24556, Turnamian, Finance, 98000).
- (b) Delete record 2.
- (c) Insert (34556, Thompson, Music, 67000).

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Initial structure of the file (Figure 13.4).

Answer

We use “ $\uparrow i$ ” to denote a pointer to record i .

(a) Insert Turnamian

header				$\uparrow 4$
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				$\uparrow 6$
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.101 The file after insert (24556, Turnamian, Finance, 98000).

Figure 1: After inserting Turnamian.

(b) Delete record 2

header				↑ 2
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2				↑ 4
record 3	22222	Einstein	Physics	95000
record 4				↑ 6
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.102 The file after delete record 2.

Figure 2: After deleting record 2.

(c) Insert Thompson

header				↑ 4
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2	34556	Thompson	Music	67000
record 3	22222	Einstein	Physics	95000
record 4				↑ 6
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.103 The file after insert (34556, Thompson, Music, 67000).

Figure 3: After inserting Thompson.

Question

Consider the relations **section** and **takes**. Give an example instance of these two relations, with three sections, each of which has five students. Then, illustrate a file structure of these relations that uses **multitable clustering**.

Answer

Example Relations:

Section Relation

Section_ID	Course	Instructor
S1	Database Systems	Smith
S2	Operating Systems	Johnson
S3	Networks	Lee

Takes Relation

Student_ID	Section_ID
101	S1
102	S1
103	S1
104	S1
105	S1
201	S2
202	S2
203	S2
204	S2
205	S2
301	S3
302	S3
303	S3
304	S3
305	S3

Multitable Clustering File Structure:

- In multitable clustering, tuples from related relations (**section** and **takes**) are stored together on the same disk block.
- Each block contains one **section** record, followed by all **takes** tuples for that section.

Example File Organization:

Block	Contents
B1	Section(S1, Database Systems, Smith); Takes(101,S1), (102,S1), (103,S1), (104,S1), (105,S1)
B2	Section(S2, Operating Systems, Johnson); Takes(201,S2), (202,S2), (203,S2), (204,S2), (205,S2)
B3	Section(S3, Networks, Lee); Takes(301,S3), (302,S3), (303,S3), (304,S3), (305,S3)

Advantages:

- Reduces disk I/O when accessing students of the same section.
- Optimizes join queries between **section** and **takes**.

Disadvantage:

- Inserts and deletions can cause reorganization when sections grow unevenly.

Question

Consider the bitmap representation of the free-space map, where for each block in the file, two bits are maintained in the bitmap. If the block is between 0 and 30 percent full the bits are 00, between 30 and 60 percent the bits are 01, between 60 and 90 percent the bits are 10, and above 90 percent the bits are 11. Such bitmaps can be kept in memory even for quite large files.

- Outline two benefits and one drawback to using two bits for a block, instead of one byte as described earlier in this chapter.
- Describe how to keep the bitmap up to date on record insertions and deletions.
- Outline the benefit of the bitmap technique over free lists in searching for free space and in updating free space information.

Answer

(a) Benefits and Drawback of Using Two Bits per Block:

- **Benefit 1: Space Efficiency** — Using 2 bits instead of 1 byte (8 bits) reduces bitmap size by a factor of 4, allowing very large files to be tracked entirely in memory.
- **Benefit 2: Granularity** — The 2-bit scheme allows approximate

tracking of block utilization in four ranges (0–30%, 30–60%, etc.), enabling better placement decisions for inserts.

- **Drawback** — Less precise than byte-based tracking; only coarse fullness levels are represented, so fine-grained space utilization decisions cannot be made.

(b) Updating the Bitmap:

- When a record is **inserted**, update the corresponding block's occupancy percentage and adjust the 2-bit value accordingly. For example, if a block's fullness increases from 55% to 70%, its bits change from 01 to 10.
- When a record is **deleted**, recompute the block's new occupancy and update the bits. For example, a decrease from 95% to 85% changes from 11 to 10.
- Such updates require modifying only a few bits in memory, making it efficient.

(c) Bitmap vs. Free List:

- **Search Efficiency** — Bitmaps allow rapid scanning (via bit operations) to locate blocks with available space, whereas free lists require pointer traversal.
- **Update Simplicity** — Modifying a bitmap requires updating a single bit pattern, while free lists may require modifying multiple block pointers.
- **Result** — Bitmaps are compact, cache-friendly, and more efficient for space management in large storage systems.

Question

It is important to be able to quickly find out if a block is present in the buffer, and if so where in the buffer it resides. Given that database buffer sizes are very large, what (in-memory) data structure would you use for this task?

Answer

A suitable data structure is a **hash table** that maps the (*file identifier, block number*) pair to its location in the buffer.

- **Reasoning:**

- Lookup time in a hash table is on average $O(1)$, which allows very fast determination of whether a block is already in the buffer.
- Each hash entry stores a reference (e.g., buffer frame index or pointer) to the block's position in memory.

- **Conclusion:** A **hash table** is the optimal in-memory structure for fast block lookup in large database buffer caches, often maintained alongside an LRU or MRU replacement policy.

Question

Suppose your university has a very large number of **takes** records, accumulated over many years. Explain how table partitioning can be done on the **takes** relation, and what benefits it could offer. Explain also one potential drawback of the technique.

Answer

Table Partitioning of the takes Relation:

- The **takes** relation can be divided (partitioned) into smaller, more manageable sub-tables called **partitions**.
- Each partition may store records based on a specific criterion, such as:
 - **Range Partitioning:** by year (e.g., 2010–2015, 2016–2020, 2021–present).
 - **List Partitioning:** by `department_id` or `course_id`.
 - **Hash Partitioning:** using a hash of `student_id` to distribute records evenly.

Benefits:

- **Improved Query Performance:** Queries on specific years or departments access only relevant partitions, reducing I/O.
- **Easier Maintenance:** Old partitions (e.g., pre-2010 data) can be archived or dropped without affecting current data.
- **Better Parallelism:** Different partitions can be processed concurrently, improving query and update throughput.

- **Faster Backup and Recovery:** Smaller partitions allow selective backup and restore operations.

Drawback:

- **Increased Management Complexity:** Partitioned tables add overhead to database design and maintenance — the optimizer, indexes, and query planner must handle multiple partitions efficiently. Poorly chosen partition keys can also lead to data skew or uneven workload distribution.

Question

Give an example of a relational-algebra expression and a query-processing strategy in each of the following situations:

- (a) MRU is preferable to LRU.
- (b) LRU is preferable to MRU.

Answer

- (a) **MRU is preferable to LRU:**

Example Relational-Algebra Expression:

$$r \bowtie s$$

(Nested-loop join between relations r and s)

Query-Processing Strategy:

- The database system uses a **block nested-loop join** where r is the outer relation and s is the inner relation.
- Each block of r is repeatedly used while scanning all blocks of s .
- If the buffer is limited, the recently used inner blocks of s can be discarded (since they will not be reused soon), while the outer block of r must remain in memory.
- **Hence, MRU (Most Recently Used)** replacement is better, because it tends to evict inner relation blocks that have just been used, preventing premature eviction of outer blocks as LRU would do.

Reasoning: In nested-loop joins, LRU mistakenly evicts outer relation pages that are still needed for multiple inner scans, causing excessive rereads. MRU avoids this issue.

(b) LRU is preferable to MRU:

Example Relational-Algebra Expression:

$$\sigma_{\text{dept}='CS'}(r)$$

(Selection on a large relation r)

Query-Processing Strategy:

- The system performs a **sequential scan** over the relation r .
- Each page is read once and not reused soon.
- LRU (Least Recently Used) is preferable here because it keeps recently accessed pages in memory, in case they are referenced again soon (e.g., in a sort or re-read phase).
- MRU, in contrast, would discard recently accessed pages, leading to possible rereads during subsequent passes.

Reasoning: For sequential scans, the access pattern benefits from LRU's tendency to keep the most recently used pages, which are likely to be needed again in short-term operations (like sorting, grouping, or merging).

Question

PostgreSQL normally uses a small buffer, leaving it to the operating system buffer manager to manage the rest of main memory available for file system buffering. Explain:

- (a) What is the benefit of this approach?
- (b) One key limitation of this approach.

Answer

(a) Benefit: Efficient Use of OS Memory Management

- By allowing the operating system to manage most of the available memory, PostgreSQL benefits from the OS's sophisticated file caching and virtual memory mechanisms.
- The OS can dynamically allocate unused memory for caching disk pages, optimizing I/O performance across multiple applications.
- This design avoids duplication of buffer management logic and allows

PostgreSQL to remain lightweight and portable across platforms.

(b) Limitation: Reduced Database-Level Control

- Since much of the caching is handled by the OS, PostgreSQL has limited visibility and control over which data pages remain in memory.
- This can lead to **double buffering** (same data cached in both PostgreSQL and the OS), wasting memory.
- Additionally, the DBMS cannot easily enforce its own replacement or prefetching policies, which may reduce performance for specialized workloads.

Question

In the variable-length record representation, a null bitmap is used to indicate if an attribute has the null value.

- (a) For variable-length fields, if the value is null, what would be stored in the offset and length fields?
- (b) In some applications, tuples have a very large number of attributes, most of which are null. Can you modify the record representation such that the only overhead for a null attribute is the single bit in the null bitmap?

Answer

(a) Handling Null Values in Offset and Length Fields:

- If a variable-length field is **null**, the system typically sets the *length* to 0 and the *offset* may either:
 - repeat the previous offset value, or
 - contain a special marker (e.g., -1) indicating “no data present.”
- The actual data portion for that attribute is omitted — the null bitmap indicates that the field should be ignored.

(b) Reducing Overhead for Many Null Attributes:

- Yes — we can modify the record structure so that **offset and length entries are stored only for non-null attributes**.
- In this design:

- The **null bitmap** indicates which attributes are null (1 bit per attribute).
- The **offset/length array** is compacted to contain entries only for non-null fields.
- During access, the system computes the correct mapping between attribute position and offset by counting preceding non-null bits.
- This approach ensures that a null attribute consumes only the **1-bit null indicator** and no additional space in the offset table.

Summary:

- (a) Null fields store no data; offset/length set to 0 or -1.
- (b) By omitting offset/length entries for null attributes, storage overhead is minimized to a single bit per attribute.

Question

Explain why the allocation of records to blocks affects database-system performance significantly.

Answer

Efficient record allocation reduces disk I/O. If related records are stored in the same block, fewer block accesses are needed, improving query speed. Poor allocation scatters data across blocks, increasing I/O and slowing performance.

Question

List two advantages and two disadvantages of each of the following strategies for storing a relational database: **(a)** Store each relation in one file. **(b)** Store multiple relations (perhaps even the entire database) in one file.

Answer

(a) Each relation in one file Advantages:

- Simple to manage and access individual relations.
- Easier to back up or recover specific tables.

Disadvantages:

- May lead to many small files, increasing file-system overhead.

- Harder to optimize cross-relation access (e.g., joins).

(b) Multiple relations in one file Advantages:

- Reduces file-system overhead and fragmentation.
- Enables efficient multi-relation access or clustering.

Disadvantages:

- Complex to manage updates or recovery of individual relations.
- Increases risk of corruption affecting multiple relations.

Question

In the sequential file organization, why is an overflow block used even if there is, at the moment, only one overflow record?

Answer

An overflow block is used to keep the physical order of the main file intact. Even if only one record overflows, a separate block prevents shifting large parts of the file for insertion and allows future overflow records to be added easily.

Question

Standard buffer managers assume each block is of the same size and costs the same to read. Consider a buffer manager that, instead of LRU, uses the rate of reference to objects, that is, how often an object has been accessed in the last n seconds. Suppose we want to store in the buffer objects of varying sizes and varying read costs (such as web pages, whose read cost depends on the site from which they are fetched). Suggest how a buffer manager may choose which block to evict from the buffer.

Answer

The buffer manager can assign a **priority score** to each object based on its:

- **Access frequency (rate of reference):** objects accessed more often are given higher priority.
- **Read cost:** objects that are expensive to fetch (high latency or cost) are given higher priority.
- **Size:** larger objects consume more buffer space and thus have lower priority

per unit of space.

Eviction Strategy: Evict the object with the **lowest cost-benefit ratio**, computed for example as:

$$\text{Score} = \frac{\text{Access Rate} \times \text{Read Cost}}{\text{Size}}$$

Objects with the lowest score are evicted first, balancing access frequency, fetch cost, and space usage.

Summary:

- Keep small, frequently accessed, high-cost-to-read objects longer.
- Evict large or rarely accessed, low-cost-to-fetch objects first.